



תכנות תחרותי

הרצאה 4



בעיית מבני נתונים

- בהינתן מערך A בגודל n , נרצה מבנה נתונים התומך בפעולות הבאות:
- $Init(A, n)$ – initialize and build the DS. $O(n)$ time.
- $Sum(i, j)$ - Returns the sum of $A_i + A_{i+1} + \dots + A_j$ (may assume $i < j$). $O(1)$ time.
- דרישת המקום למבנה: $O(n)$. (כיאה לתכנות תחרותי, נרצה גם קבועים נמוכים ככל הניתן).

פתרון בעיית מבני נתונים

• ניתן לשמור את הסכום של כל רישא במערך בגודל $n + 1$ המקיים:

$$P[0] = 0, \quad P[i] = P[i - 1] + A[i - 1] \quad \forall i \geq 1$$

ובזאת סיימנו את הבנייה.

בוודאי ש:

$$Sum(i, j) = P[j] - P[i - 1]$$

טוויסט בעלילה

- שימו לב שהפתרון הקודם עובד כל כך טוב רק בגלל שהמערך לא משתנה. אם ננסה לשנות את המערך A במיקום i ניאלץ לעשות $n - i + 1$ שינויים למערך P וזו אינה דרך חכמה לעשות זאת.
- אז שינוי לבעיה המקורית יגרור באולי סיבוכויות שונות לשאר הפעולות.
- מה אם נאפשר פקטור נוסף של $\log n$?

בעיית מבני נתונים 2

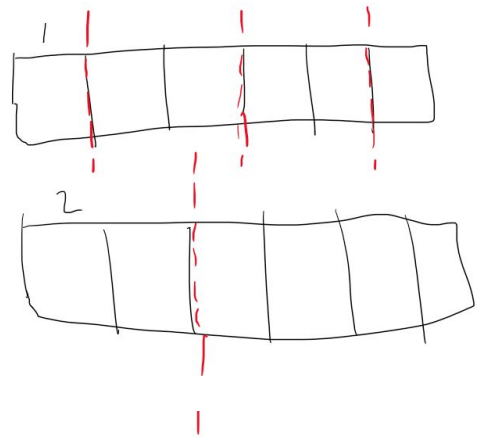
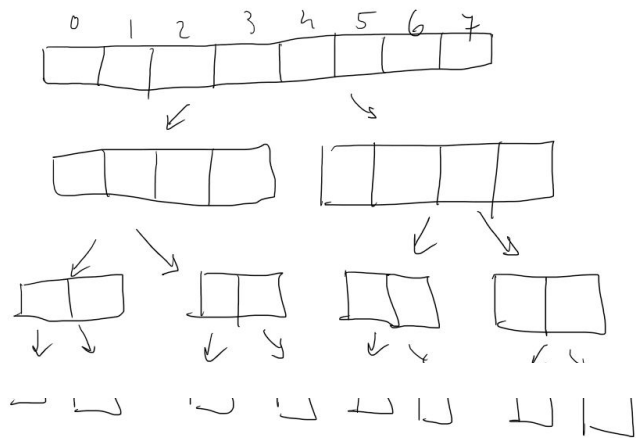
- בהינתן מערך A בגודל n , נרצה מבנה נתונים התומך בפעולות הבאות: (מתנצלים מראש על הלעז, הרבה יותר קריא)
 - $Init(A, n)$ – initialize and build the DS. $O(n)$ time.
 - $Sum(i, j)$ - Returns the sum of $A_i + A_{i+1} + \dots + A_j$ (may assume $i < j$). $O(\log n)$ time.
 - $Change(i, k)$ - Changes $A[i] += k$. $O(\log n)$ time.
- דרישת המקום למבנה: $O(n)$. (כיאה לתכנות תחרותי, נרצה גם קבועים נמוכים ככל הניתן).

אבחנה ראשונה

- טענה: לכל $i < j$ ניתן לפרק את $[i, j]$ ל- $O(\log n)$ מקטעים שכל אחד הוא בגודל חזקת 2.
- הסבר: נביט על $k = j - i + 1$, בפרט $(k)_2$ כלומר הייצוג הבינארי של k . ניתן לפרק את k לגדלים של חזקות 2 ע"פ אם הביט דולק או לא. (להוכחה פורמלית יותר – אינדוקציה חזקה).
- $(14)_2 = 1110 \Rightarrow 14 = 2^3 + 2^2 + 2^1 + 0 = 8 + 4 + 2 + 0$

אבחנה שנייה

- טענה: יתרה מכך אותו k יכול להתפרק ל- $O(\log n)$ חזקות 2 בסדר כך שכל בלוק בגודל B מתחיל באינדקס l המקיים $l \equiv 0 \pmod{B}$.
- הוכחה: לא רלוונטי לנו





sort the
queries /
sparse table
/ fenwick tree



segtree

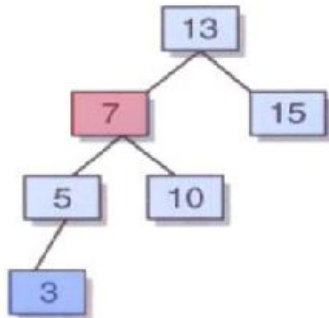


עץ סגמנטים

Segment Tree

הרעיון

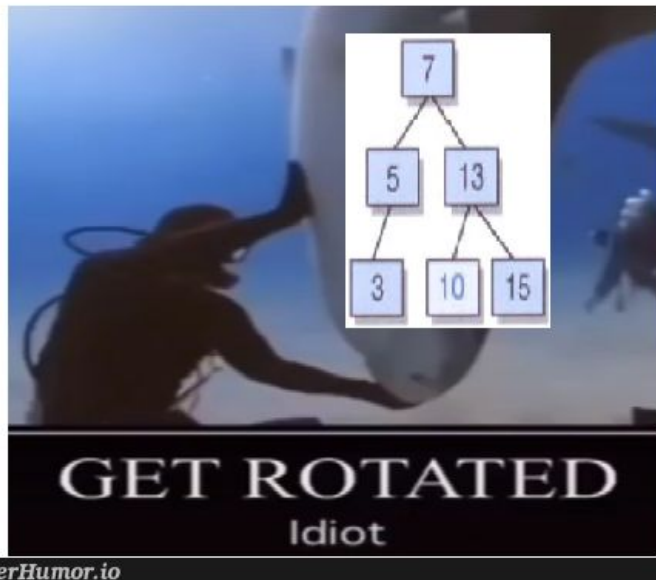
The BST:



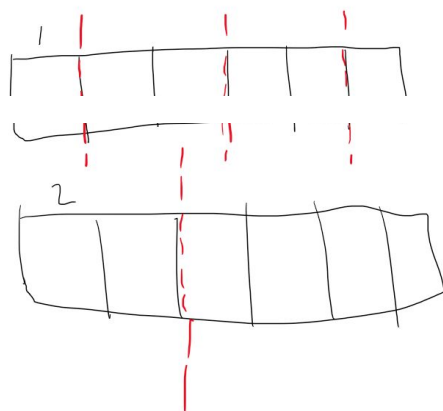
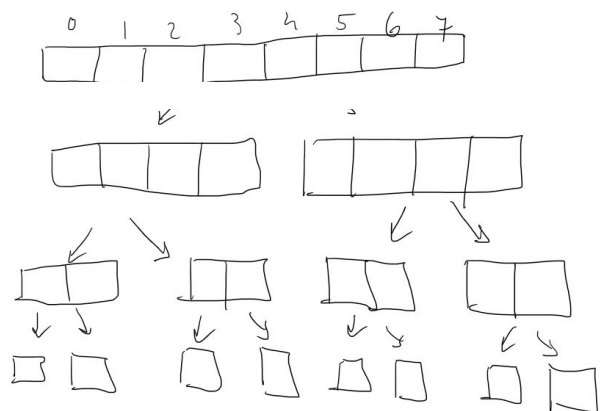
• נשתמש במבנה נתונים שהוא מייצג בדיוק את העץ מאבחנה 2.

כלומר משהו כזה:

Me:



הרעיון

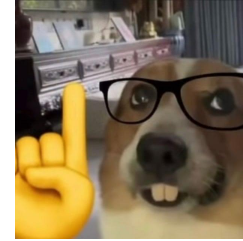


- נשתמש במבנה נתונים שהוא מייצג בדיוק את העץ מאבחנה 2.

- באופן זה, כל שינוי שאנחנו נעשה ישפיע

רק על $O(\log n)$ ערכים (שנמצאים במסלול לסינגלטון של האינדקס), וכל שאילתא תהיה מורכבת רק מ- $O(\log n)$ איברים שאת כולם אנחנו חישובנו כבר.

מבחינת זמן בנייה ומקום – לאחסן את העץ ייקח $2n$ צמתים, ולבנות אותו ייקח $O(n)$ שכן עוברים על כל צומת פעמיים – לחישוב הערך שלו, ולחישוב הערך של אבא שלו



למעשה

- במהלך כל האבחנות שעשינו לא הזכרנו את השימוש בסכום אפילו לא פעם אחת – האם התבלבלנו?
- עצי סגמנטים הם כלי מאוד חזק – שעובד על כל פעולה אסוציאטיבית (כלומר כזו שאפשר להעלים לה את הסוגריים בלי לשנות את התוצאה $((a + b) + c = a + (b + c))$).
- הם כלי כל כך חזק שאפילו זה לא תנאי הכרחי, מספיק לדעת איך לחבר בזמן טוב את הפתרון לשני מקטעים סמוכים.

Code No.1

```
vi a;
struct sum_segment_tree{
    int l, r, m;
    struct sum_segment_tree *left, *right;
    int value;

    sum_segment_tree(int L, int R) : l(L), r(R), m((l+r)/2), left(nullptr), right(nullptr){
        if(l < r) {
            left = new sum_segment_tree(L, R: m);
            right = new sum_segment_tree(L: m+1, R);
            value = left->value + right->value;
        }
        else if (l == r){
            value = a[l];
        }
    }

    void update(int i, int k){
        value += k;
        if(l == r) return;
        if(m < i) right->update(i, k);
        else left->update(i, k);
    }

    int query(int L, int R){
        if(R < l || r < L) return 0; //
        if(L <= l && r <= R) return value;
        return left->query(L, R) + right->query(L, R);
    }
};
```

בעיית מבני נתונים 3

- בהינתן מערך A בגודל n , נרצה מבנה נתונים התומך בפעולות הבאות: (מתנצלים מראש על הלעז, הרבה יותר קריא)
 - $Init(A, n)$ – initialize and build the DS. $O(n)$ time.
 - $Sum(i, j)$ - Returns the sum of $A_i + A_{i+1} + \dots + A_j$ (may assume $i < j$). $O(\log n)$ time.
 - $Change(i, j, k)$ - Changes $A[i, \dots, j] += k$. $O(\log n)$ time.
- דרישת המקום למבנה: $O(n)$. (כיאה לתכנות תחרותי, נרצה גם קבועים נמוכים ככל הניתן).

פתרון בעיית מבני נתונים 3

- עכשיו נוכל לשוב לפרק את התחום $[i,j]$ למקטעים שחישבנו את התוצאה, ונשים לב שבהינתן טווח שכולו נמצא בתוך טווח העדכון – אני יודע בוודאות שהסכום שלו גדל ב- $k \cdot (r - l + 1)$, ולכן זהו השינוי שנצטרך לבצע לקוד.

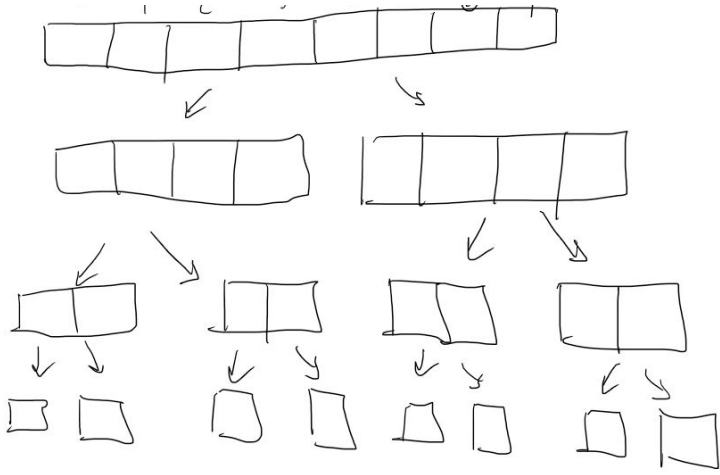
```
void update(int L, int R, int k){  
    if(R < l || r < L) return;  
    if(L <= l && r <= R) value += k * (r-l+1);  
    else left->update(L, R, k), right->update(L, R, k);  
}
```

- המבנה הזה מאוד נוח והוא גם מאוד קל ואינטואיטיבי לכתובה!

סתם לא באמת, שיקרנו

כמעט אבל!

לא על הכל אבל



- אז תחשבו מה קורה אם עדכנו מקטע גדול, ואז היינו צריכים את הילדים שלו.
- כאן מגיעה הטעות – אבל גם השינוי במחשבה.

- מה אם נפרק את החישוב של כל ערך לכדי חישוב של סכום המסלול שלו?
- כך נוכל לתמוך בשאילתות על מספר אחד, אבל לאפשר נכונות בעדכונים של טווחים:

```
sum_segment_tree(int L, int R) : l(L), r(R), m((l+r)/2), left(nullptr), right(nullptr){
    if(l < r) {
        left = new sum_segment_tree(L, R: m);
        right = new sum_segment_tree(L: m+1, R);
        value = 0;
    }
    else if (l == r){
        value = a[l];
    }
}
```

```
void update(int L, int R, int k){
    if(R < l || r < L) return;
    if(L <= l && r <= R) value += k;
    else left->update(L, R, k), right->update(L, R, k);
}

int get(int i){
    if(l == r) return value;
    return value + (m < i ? right->get(i) : left->get(i));
};
```


אבל רגע

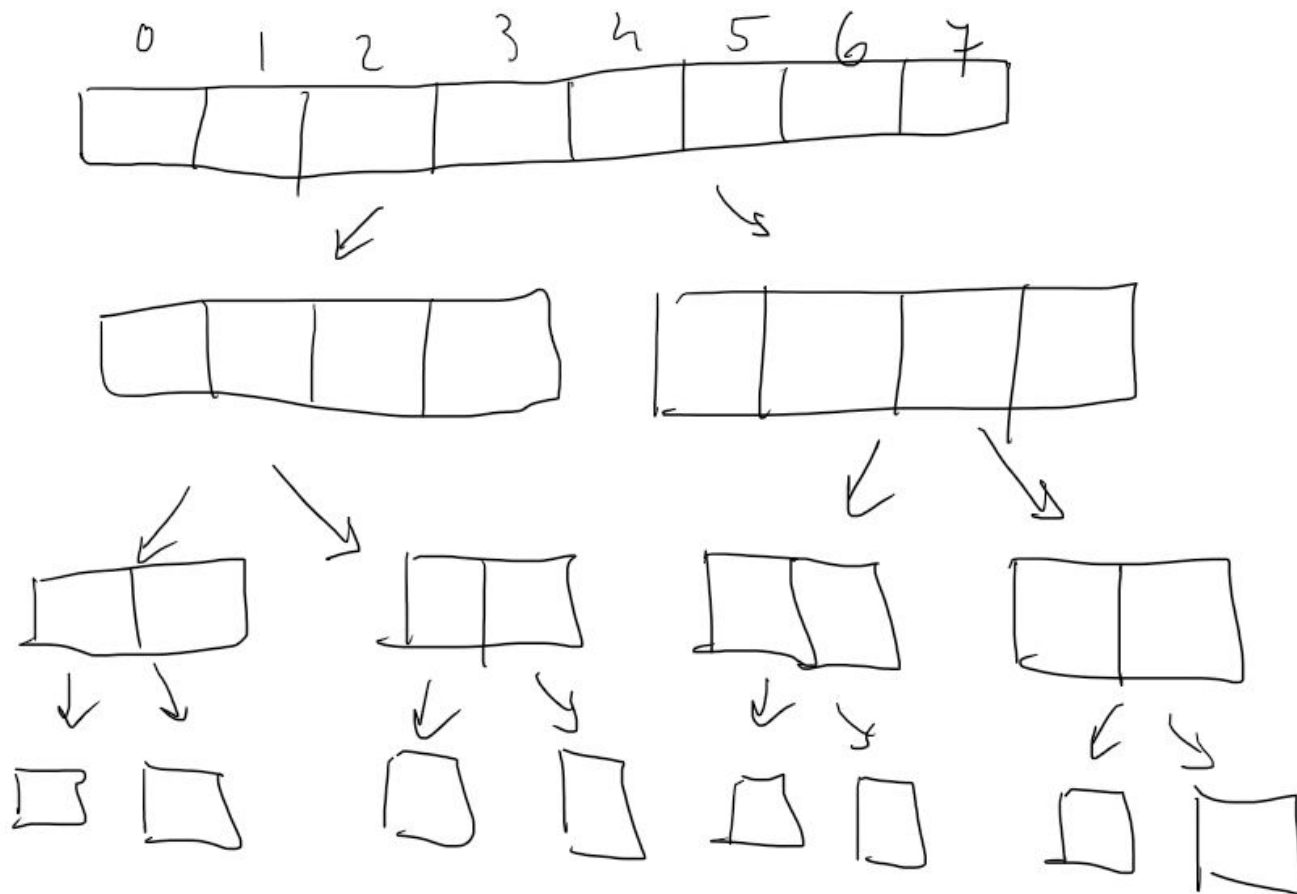
- אנחנו רוצים עדכונים על טווחים, וגם שאלות על טווחים!
- מה שראינו עכשיו מאפשר רק שאלות נקודתיות



Lazy propagation

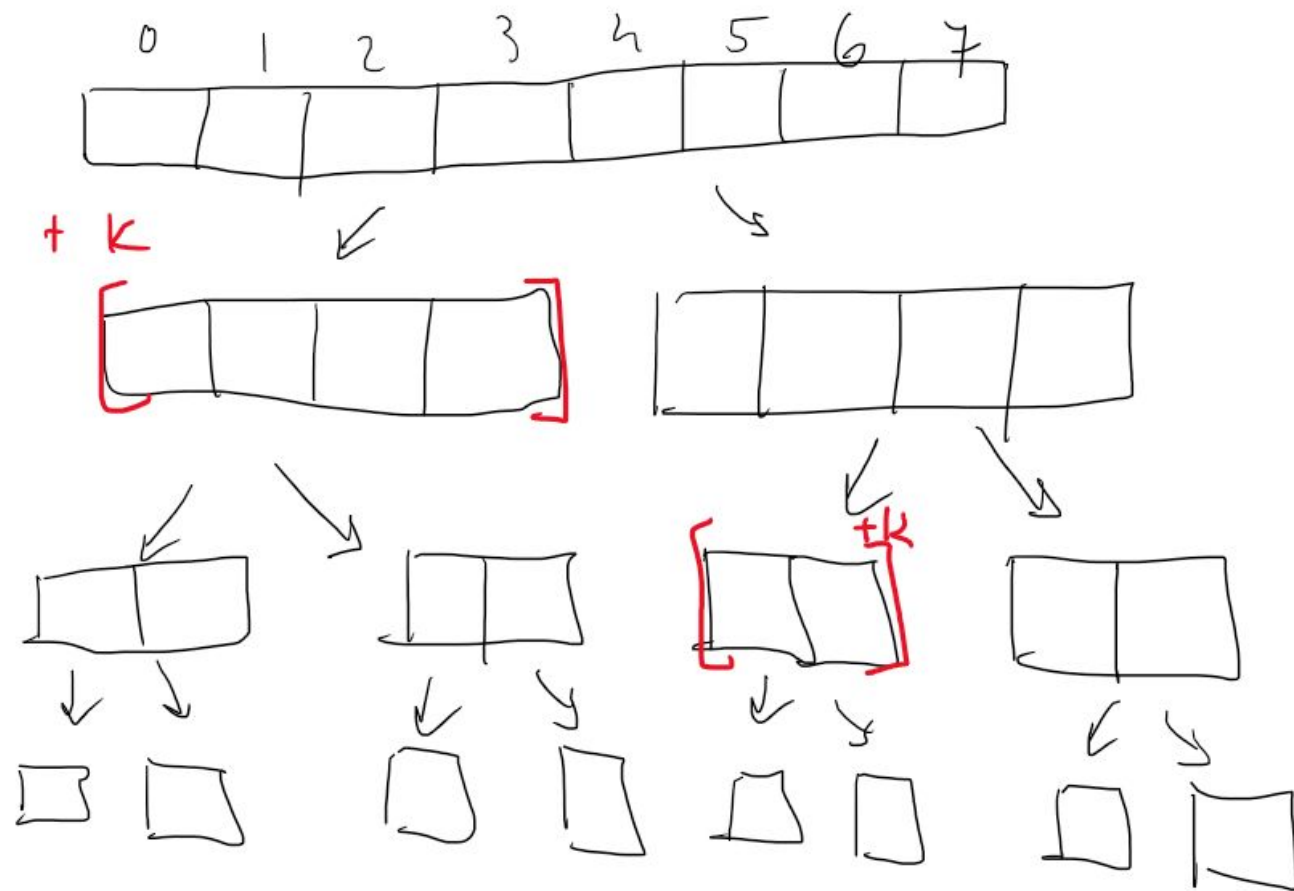
Segment Tree Kashe

עצלות לשמה



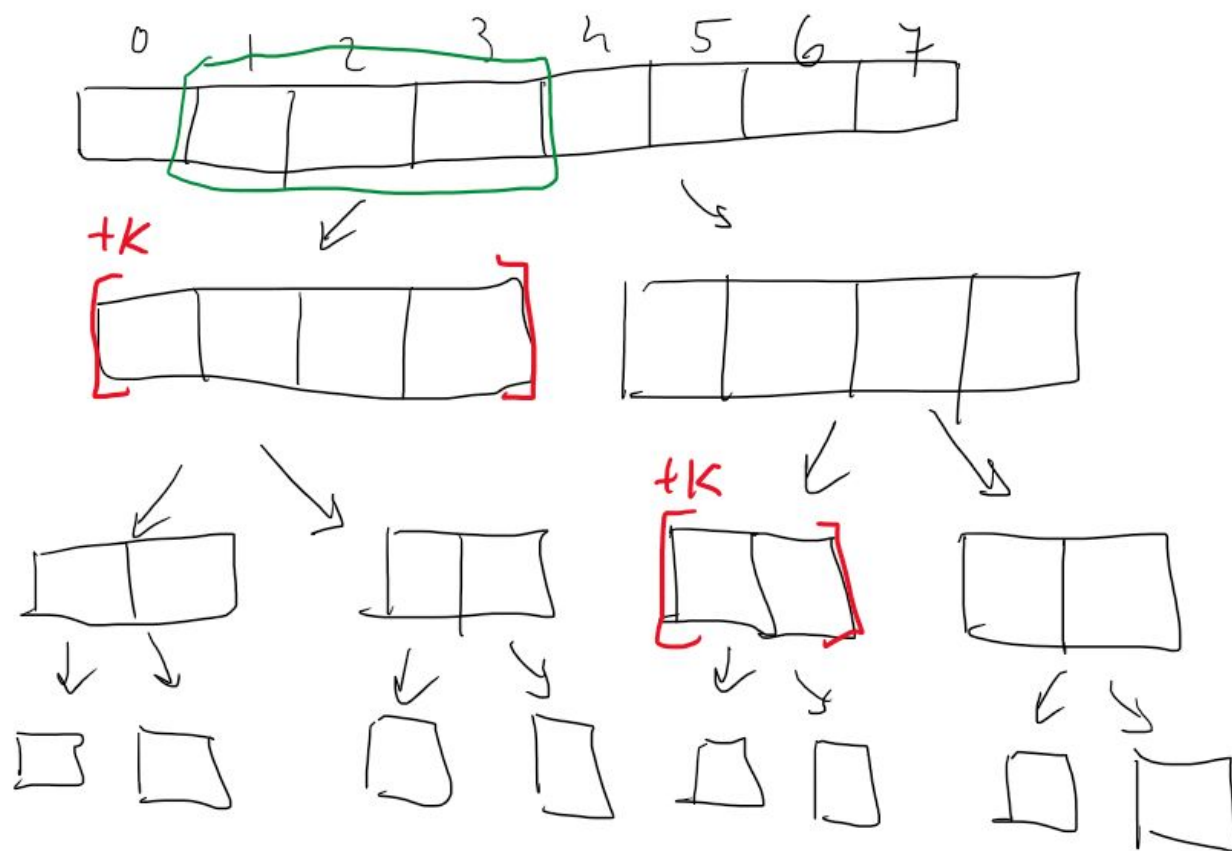
Update(0, 5, K)

עצלות לשמה



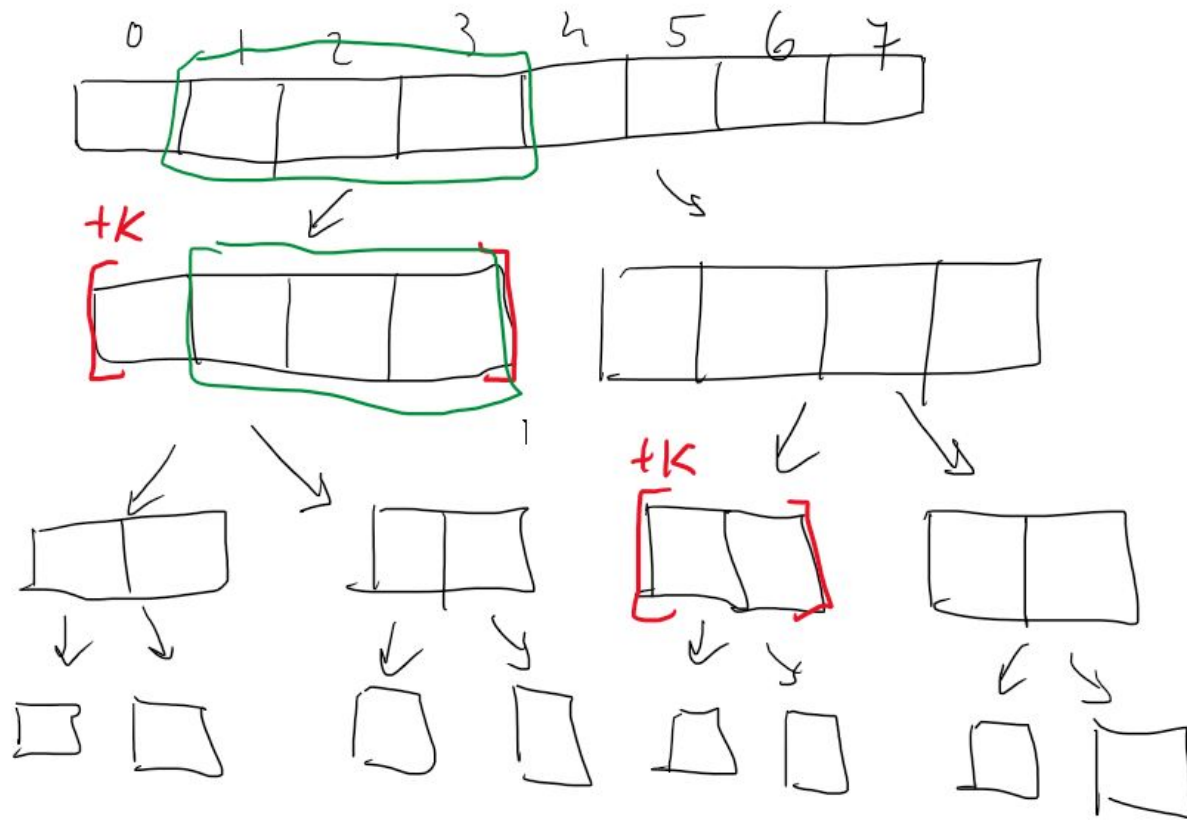
עצלות לשמה

$\text{sum}(1, 3)$



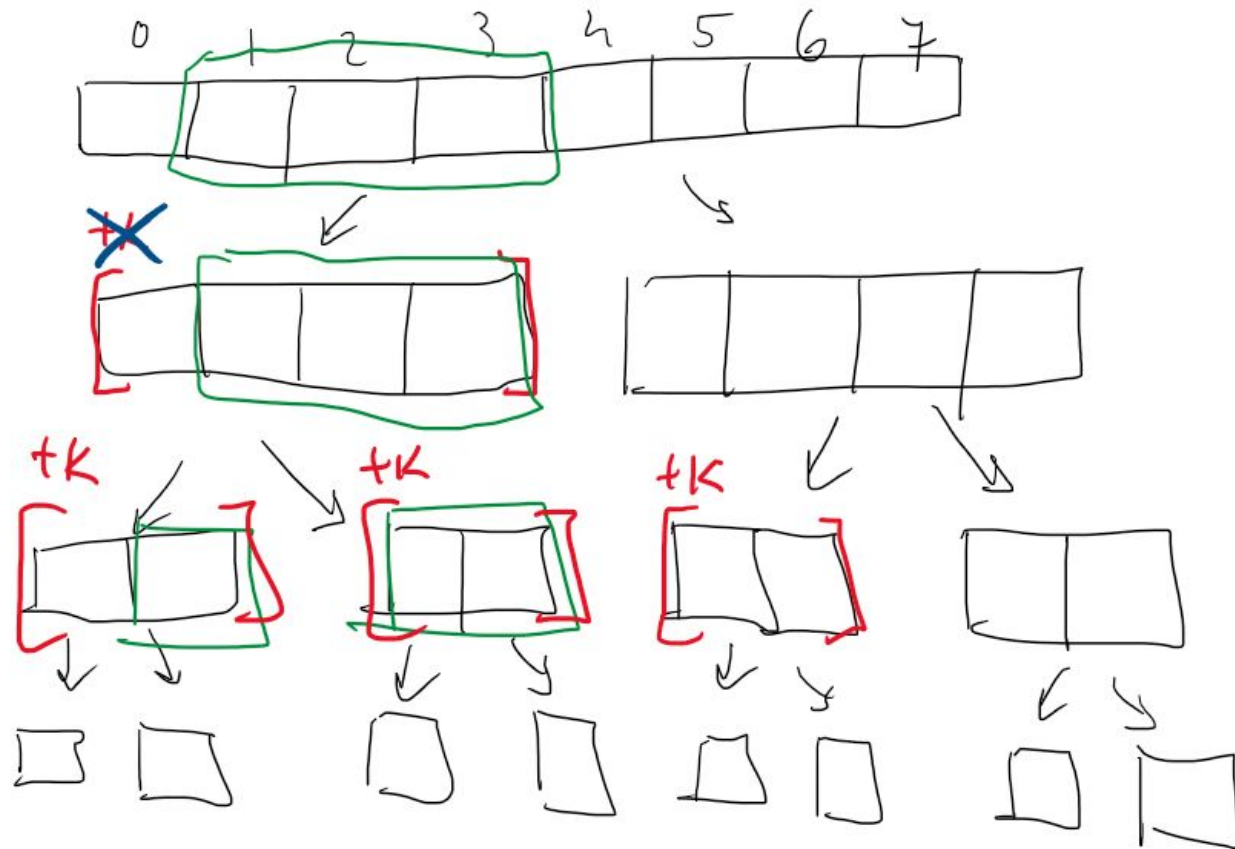
עצלות לשמה

sum (1, 3)



עצלות לשמה

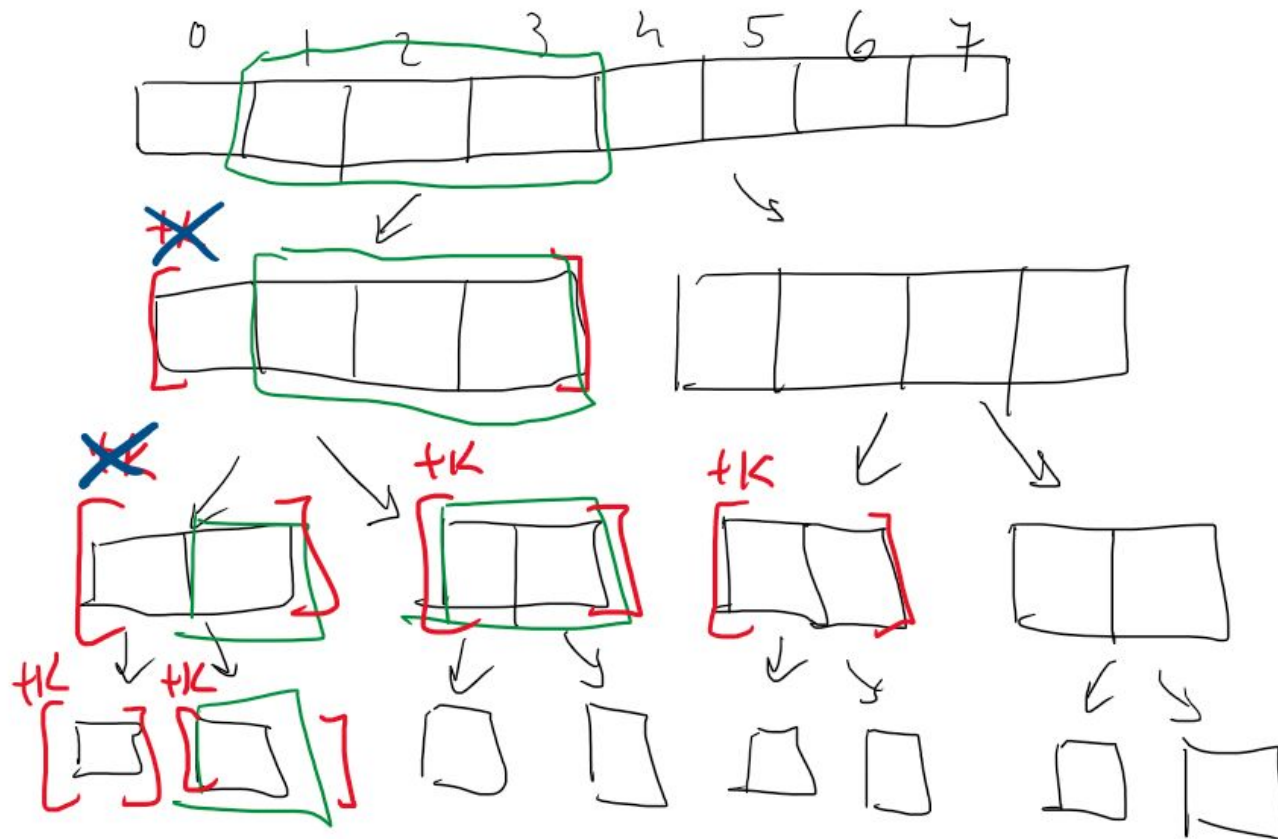
$\text{sum}(1, 3)$



עצלות לשמה

$\text{sum}(1, 3)$

6



Push down

- על כל פעולה שנעשה, נרצה לדחוף את העדכון לבנים תוך עדכון הצומת הנוכחי.
- צריך לזכור לאפס את העדכון של האב, ולשמור ערך בשם lazy שיהיה "ערך השינוי של הטווח בצומת הזה".
- כך הפעולה של דחיפה למטה תתבצע ב $O(1)$ ע"י לעדכן את lazy של צד ימין ושל צד שמאל, ואתחול של הנוכחי.
- פעולה זו תיקח $O(1)$ וסה"כ עוברים בעץ ב $O(\log n)$ צמתים ולכן הסיבוכיות לפעולות נשארת $O(\log n)$.

```
void pushdown(){
    left->value += (m-l+1) * lazy; // [l, m] is m-l+1
    left->lazy += lazy;
    right->value += (r-m) * lazy; // [m+1, r] is r-m
    right->lazy += lazy;
    lazy = 0;
}
```

- הקוד לכך:

Pull Up



- בדרך חזרה ברקורסיה, חשוב לוודא שאנחנו משתמשים בvalue שהוא נכון ומעודכן – ולכן נמשוך למטה מהבנים שלו את הvalue שלהם ונחשב את של הצומת הנוכחי.
- בכניסה לרקורסיה נדחוף למטה, וביציאה ממנה נחשב את הערך של האב מהבנים שלו – במילים אחרות נדחוף למטה לפני שנרד למטה, נמשוך למעלה לפני שנעלה למעלה.

```
void pullup(){  
    value = left->value + right->value;  
}
```

השינויים הנוספים

- אז הוספנו שדה של lazy. בנוסף שינינו את sum, update באופן הבא:

```
void update(int L, int R, int k){
    if(R < l || r < L) return;
    if(L <= l && r <= R) {
        value += k * (r-l+1);
        lazy += k;
        return;
    }
    pushdown();
    left->update(L, R, k);
    right->update(L, R, k);
    pullup();
}
```

```
int sum(int L, int R){
    if(R < l || r < L) return 0;
    if(L <= l && r <= R) return value;
    pushdown();
    int x = left->sum(L, R);
    int y = right->sum(L, R);
    // pullup();
    return x+y;
}
```

כאמור – זה מבנה נתונים מאוד חזק

- אפשר לתמוך הרבה פעולות, והזמן הוא worst-case , לא לשיעורין.
- אפשר לתמוך בהרבה פעולות, והרבה פעמים גם במקביל (כלומר גם עדכונים על טווחים כתוספת וגם כהשמה).

- Sum
- Max
- Min
- GCD
- $[l, r] = k$
- $[l, r] += k$
- XOR
- AND
- OR
- etc/.

בעיית מבני נתונים 2

- בהינתן מערך A בגודל n , נרצה מבנה נתונים התומך בפעולות הבאות: (מתנצלים מראש על הלעז, הרבה יותר קריא)
 - $Init(A, n)$ – initialize and build the DS. $O(n)$ time.
 - $Sum(i, j)$ - Returns the sum of $A_i + A_{i+1} + \dots + A_j$ (may assume $i < j$). $O(\log n)$ time.
 - $Change(i, k)$ - Changes $A[i] += k$. $O(\log n)$ time.
- דרישת המקום למבנה: $O(n)$. (כיאה לתכנות תחרותי, נרצה גם קבועים נמוכים ככל הניתן).

פתרון בעיית מבני נתונים 2 אבל אחרת

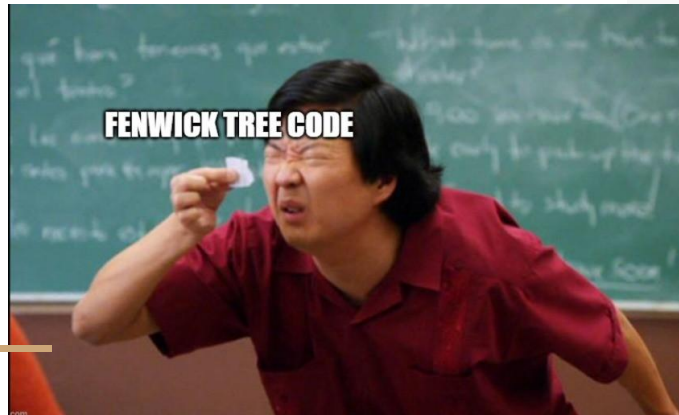
- אולי נאחסן את הסכומים בצורה שונה, כלומר:

$$T_i = \sum_{j=g(i)}^i a_j$$

דבר כזה יכול להראות משהו כמו:

$$a_0, a_0 + a_1, a_2, a_0 + a_1 + a_2 + a_3$$

ברור שמספיקים לנו n משתנים בלתי תלויים כדי להרכיב את הסכומים הנכונים באיזושהי דרך, השאלה היא איך בצורה נוחה ויעילה?



10000
01111
01110
01101
01100
01011
01010
01001
01000
00111
00110
00101
00100
00011
00010
00001

Fenwick Tree

BIT = Binary Indexed tree

הגדרת $g(i)$

- ניתן לבחור כל מיני $g(i)$ ועבור חלק מהן המבנה יעבוד בסדר. במגבלות הקורס נעסוק ב $g(i)$ ספציפית:
 - $g(11) = g(1011_2) = 1000_2$
 - $g(10) = g(1010_2) = 1010_2$
 - $g(7) = g(111_2) = 000_2$
-
- מה הדפוס?
-
- $g(i) = \begin{cases} i & \text{if } LSB(i) = 0 \\ i \text{ w/o leading 1s} & \text{else} \end{cases}$

מתברר ש...

- ניתן לאחסן את הסכומים האלה ע"י משחקי ביטים חכמים: (לא באמת רלוונטי להיכנס לפרטים).

```
struct FenwickTree {
    vector<int> bit; // binary indexed tree
    int n;
    FenwickTree(int n) {
        this->n = n;
        bit.assign(n, val: 0);
    }
    FenwickTree(vector<int> const &a) : FenwickTree( n: a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            add( idx: i, delta: a[i]);
    }
}
```

```
int sum(int r) {
    int ret = 0;
    for (; r >= 0; r = (r & (r + 1)) - 1)
        ret += bit[r];
    return ret;
}

int sum(int l, int r) {
    return sum(r) - sum( r: l - 1);
}

void add(int idx, int delta) {
    for (; idx < n; idx = idx | (idx + 1))
        bit[idx] += delta;
}

};
```



אבל בעצם

- למה שנשתמש בזה אם אנחנו כבר יודעים seg tree?
- קוד הרבה יותר קריא, ומכוון בדיוק למה שצריך. בנוסף העובדה שסוכמים רישא מאפשרת לעשות רעיונות מאוד יפים (ויותר קל להסביר עם מבנה חדש).
- אז recap קצר על הAPI (פונקציות של פנוויק):
 - $Add(i, k) - A[i] += k$
 - $Sum(j) - \text{return sum of prefix } j$

שאלות נקודה ועדכוני טווח

- אנחנו יודעים לבצע שאלות טווח ועדכוני נקודה. מה אם נשתמש בסכום בשביל הערך, ובערך בשביל הסכום?

- נאתחל את כל המערך ל-0.

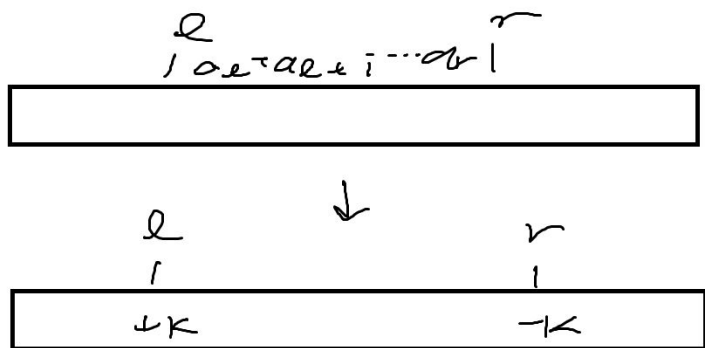
- כשנרצה לחשב ערך במיקום i נחשב אותו ע"י הסכום של $a_0 + \dots + a_i$.

- כשנרצה לעדכן טווח ולהוסיף לו k , נעשה זאת ע"י להוסיף k לאינדקס של

- l ולהוריד k מהאינדקס של $r + 1$.

- ניתן לתחזק אפילו שני עצי פנוויק, אחד מאותחל ל-0 ויטפל בעדכונים,

אחר מאותחל למערך ויטפל בבקשות הסכום. ניתן לתחזק את זה אבל לא במסגרת הקורס.



שאלת תכנות תחרותי

- בשדרה הראשית בעיר יש n בתי מלון, בכל אחד מספר כלשהו h_i של חדרים פנויים. לשדרה מגיעים m אוטובוסים עם תיירים עליהם וכל אוטובוס צריך r_i חדרי מלון, כל החדרים צריכים להיות באותו בית מלון.
- האוטובוסים מגיעים לפי הסדר וכל אוטובוס עוצר בבית המלון הראשון שיש בו מספיק חדרים, מוריד את הנוסעים ובבית המלון הזה יש עכשיו r_i פחות חדרים פנויים.
- מצאו לכל אוטובוס באיזה בית מלון הוא עוצר או אם זה בלתי אפשרי.

שאלת תכנות תחרותי - פתרון

- נתחזק segment tree של מקסימום, כל צומת ידע את מספר החדרים הפנויים המקסימלי באחד מבתי המלון שנמצאים תחתיו (כלומר צומת עבור הטווח $[i,j]$ יודע את מספר החדרים הפנויים המקסימלי בבתי המלון $[i,j]$)
- כדי למצוא את בית המלון הראשון (הכי שמאלי) עם מספיק חדרי מלון פנויים r_i , נתחיל מהשורש, קודם כל נבדוק אם הערך של השורש גדול שווה r_i . אם הערך בבן השמאלי גדול שווה r_i נרד עם השאילתה שמאלה אחרת ימינה.
- כדי לעדכן ערך של בית מלון מסויים, נעדכן את הערך שלו ואז לכל צומת במסלול מהשורש אליו כאשר עולים בקריאות בעץ נעדכן את הערך להיות המקסימום בין הערך של שני הבנים שלו.

שאלה נוספת

- נתון מערך A מגודל n של מספרים חסומים ב- n ונתונות q שאילתות טווחים $[i, j]$
- כל שאילתה שואלת מה מספר הערכים **השונים** בטווח $[i, j]$

שאלה נוספת - פתרון

- נשתמש בsegment tree
- כל צומת יחזיק במספר האיברים השונים בתת העץ שלו
- כדי לחבר צמתים פשוט אמ... אמ...



בעיה

- מה קורה אם אנחנו לא יודעים לחבר סגמנטים בזמן מהיר?



instrucko

Word of the week

Flabbergasted

Extremely surprised or shocked



Example: When he saw the giant robot ,
he was completely flabbergasted

Mo's Algorithm – sqrt decomp

- אם אנחנו לא יודעים לחבר סגמנטים בזמן מהיר אבל יכולים להוסיף ולהוריד איבר אחד בזמן מהיר
 - למשל בדוגמה ממקודם, נשמור כמה ראינו מכל איבר וכמה איברים שונים ראינו, כדי להוסיף או להוריד איבר אחד לטווח נוכל לעשות את זה בקלות ב- $O(1)$
- וגם אנחנו מקבלים את כל השאילתות מראש (offline) נוכל להשתמש באלגוריתם של מו

Mo's Algorithm – sqrt decomp



- הרעיון של האלגוריתם הוא כמעט לבצע פשוט את כל השאילתות על ידי מעבר על כל הטווח.

- היופי באלגוריתם הוא הסדר שבו נעבור על השאילתות!

- כאשר אנחנו עוברים מלחשב את השאילתה $[l_1, r_1]$ לחישוב השאילתה $[l_2, r_2]$ אנחנו פשוט נוסיף ונוריד אחד אחד את כל האיברים שצריך בשמאל ובימין.

- אבל באיזה סדר לעבור על השאילתות?

Mo's Algorithm – sqrt decomp

- כמו שאולי אפשר להבין מהשם, אנחנו נחלק את המערך למקטעים באורך $\sqrt{n}$ כמובן שיש $\sqrt{n}$ כאלו.
- את השאילתה $[l, r]$ אנחנו נמיין קודם כל לפי ה**בלוק** שבו נמצא הצד השמאלי של השאילתה כלומר $\frac{l}{\sqrt{n}}$ ואז לפי המיקום של הצד הימני כלומר r .

```
sort(all(queries), [&](Query x, Query y) {  
    int block_x = x.l / block_size;  
    int block_y = y.l / block_size;  
  
    if (block_x != block_y)  
        return block_x < block_y;  
  
    if (block_x % 2 == 0)  
        return x.r < y.r;  
  
    return x.r > y.r;  
});
```

- זהו זה הכל!!



Mo's Algorithm – sqrt decomp



למה זה עובד אבל?

Mo's Algorithm – sqrt decomp



- במקום להסתכל עבור כל שאילתה כמה זמן היא לקחה, נסתכל במקום עבור כל השאילתות יחד כמה פעמים הגדלנו והקטנו את הטווח שלנו ימינה וכמה פעמים הגדלנו והקטנו את הטווח שלנו שמאלה.
- עבור כל בלוק של $\sqrt{n}$, בגלל שהשאילתות בתוך הבלוק ממיינות לפי הצד הימני שלהן, אנחנו רק נגדיל את הצד הימני של הטווח שעליו אנחנו מסתכלים. לכן עבור כל בלוק נגדיל את הצד הימני לכל היותר n פעמים, סך כל הבלוקים נבצע $n\sqrt{n}$ הגדלות ימינה.
- כל שאילתה תגדיל/תקטין את הצד השמאלי של השאילתה לכל היותר $\sqrt{n}$ כי אנחנו יודעים שאנחנו נישאר בתוך הבלוק. סך כל השאילתות יבצעו $q\sqrt{n}$ הגדלות/הקטנות של הצד השמאלי של הטווח.

Mo's Algorithm – sqrt decomp

- סך הכל נבצע לכל היותר $n\sqrt{n}$ הגדלות של הטווח כלפי ימין ולכל היותר $q\sqrt{n}$ הגדלות/הקטנות של הטווח כלפי שמאל.
- סיבוכיות הזמן של הפתרון $O((q + n)\sqrt{n})$
- חשוב לזכור שני דברים
 - האלגוריתם עובד offline כלומר אנחנו מקבלים את כל השאילתות מראש, אם היינו צריכים לענות על שאילתה אחת ורק אז מספרים לנו מה השאילתה השנייה, לא היינו יכולים להשתמש בו.
 - האלגוריתם ממיין את השאילתות, לכן לא נוכל לעדכן את המערך באמצע, לפחות לא בלי לעשות שינויים לאגלוריתם.

שאלה נוספת

- נתון מערך A מגודל n של מספרים חסומים ב- n ונתונות q שאילתות טווחים $[i, j]$
- כל שאילתה שואלת מה מספר הערכים **השונים** בטווח $[i, j]$

שאלה נוספת - פתרון

- נתון מערך A מגודל n של מספרים חסומים ב- n ונתונות q שאלות טווחים $[i, j]$
- כל שאלתה שואלת מה מספר הערכים **השונים** בטווח $[i, j]$
- עכשיו הבעיה ממש פשוטה, נתשמש באלגוריתם של מו. נחזיק מערך ובו כמות המופעים בטווח עליו אנחנו מסתכלים כרגע ובנוסף משתנה של מספר הערכים השונים בטווח. כשמקטינים ומגדילים את הטווח פשוט מקטינים או מגדילים את התא המתאים במערך ב-1 ואם התא עכשיו שווה 0 או כבר לא שווה 0 נקטין או נגדיל את המונה של מספר הערכים השונים בטווח בהתאמה.